Lid-Driven Cavity Flow at $\mathrm{Re} = 100$

Incompressible Navier–Stokes Solution with the SMAC Projection Method on a Staggered Grid (C)

Course Project 5 – AM5630 Foundation of Computational Fluid Dynamics (Computer Assignment 3, Jan–May 2024)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras

Revised report, October 2026

Abstract

Steady two-dimensional incompressible flow in a unit square cavity driven by a sliding lid is computed at $\mathrm{Re} = 100$ with the Simplified Marker-And-Cell (SMAC) fractional-step method on a staggered (MAC) grid of 99×99 cells. The original assignment code fixed the program structure (constants, `compute_pressure`, `update_velocity`, `compute_vorticity`, time loop and output files) but did not solve the governing equations and terminated with a segmentation fault; the function bodies were completed here following the SMAC algorithm stated in the original report. Time marching ($\Delta t = 10^{-3}$, CFL $= 0.099$) reaches steady state ($\max|u^{n+1} - u^n| < 10^{-7}$) at $t = 13.56$. The primary-vortex stream function $\psi_{min} = -0.10337$ and centre $(0.616, 0.737)$ agree with the benchmark of Ghia, Ghia & Shin (1982) [-0.10342 at $(0.617, 0.734)$] to within 0.05% and one cell, the centreline velocities agree to within $\max|\Delta u| = 0.005$ and $\max|\Delta v| = 0.009$, and both secondary corner eddies are resolved and converge towards the benchmark strengths under grid refinement.

Contents

1 Problem definition **2**

2 Numerical method **2**

2.1 Staggered (MAC) grid . . . 2

2.2 SMAC fractional step . . . 2

2.3 Stability and accuracy . . . 3

2.4 Algorithm and flowchart . . . 3

3 Code review of the original submission **4**

4 Results **5**

4.1 Convergence to steady state . . . 5

4.2 Flow structure . . . 5

4.3 Validation against Ghia et al. (1982) . . . 6

5 Conclusions **7**

A Source code **7**

1 Problem definition

A unit square cavity is filled with an incompressible Newtonian fluid. The top wall (lid) slides at $U = 1$; the other three walls are at rest. With L and U as scales, the non-dimensional equations are

$$\nabla \cdot \mathbf{u} = 0, \tag{1}$$

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u}\mathbf{u}) = -\nabla p + \frac{1}{\mathrm{Re}} \nabla^2 \mathbf{u}, \qquad \mathrm{Re} = \frac{UL}{\nu} = 100, \tag{2}$$

with $\mathbf{u} = (u, v)$, no slip on all walls, $u = 1$, $v = 0$ on the lid, and $\mathbf{u} = 0$ at $t = 0$. Pressure needs no physical boundary condition; the projection step imposes $\partial\phi/\partial n = 0$ on the pressure correction, which is consistent with $\mathbf{u}\cdot\mathbf{n} = 0$. The steady state is obtained as the long-time limit of the transient problem.

2 Numerical method

2.1 Staggered (MAC) grid

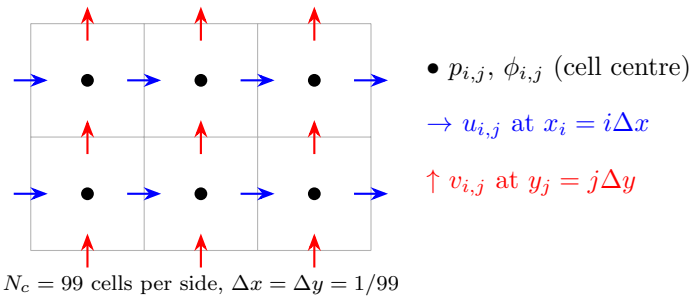


Figure 1: MAC arrangement. Velocities on faces and pressure at centres couple pressure and velocity on adjacent points, which prevents the odd–even (checkerboard) pressure decoupling of a collocated central scheme.

Wall velocities normal to a wall lie on the wall and are set directly ($u = 0$ on $x = 0, 1$; $v = 0$ on $y = 0, 1$). Tangential velocities are imposed through a ghost layer by linear extrapolation:

$$u_{i,0} = -u_{i,1}, \qquad u_{i,N_c+1} = 2U - u_{i,N_c}, \qquad v_{0,j} = -v_{1,j}, \qquad v_{N_c+1,j} = -v_{N_c,j}. \tag{3}$$

2.2 SMAC fractional step

Each time step $t^n \to t^{n+1}$ consists of:

1. **Predictor** with the old pressure (explicit Euler; convective terms in conservative form, second-order central averages to the faces; five-point viscous Laplacian):

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t \Big[-\nabla \cdot (\mathbf{u}\mathbf{u})^n + \frac{1}{\mathrm{Re}} \nabla^2 \mathbf{u}^n - \nabla p^n \Big]. \tag{4}$$

   For example, at the u-face (i, j): $\big[\partial(u^2)/\partial x\big]_{ij} = (\bar{u}_e^2 - \bar{u}_w^2)/\Delta x$, $\big[\partial(uv)/\partial y\big]_{ij} = (\bar{u}_n \bar{v}_n - \bar{u}_s \bar{v}_s)/\Delta y$, with bars denoting two-point averages.

2. **Pressure correction**: requiring $\nabla \cdot \mathbf{u}^{n+1} = 0$ for $\mathbf{u}^{n+1} = \mathbf{u}^* - \Delta t \nabla \phi$ gives the Poisson equation

$$\nabla^2 \phi = \frac{1}{\Delta t} \nabla \cdot \mathbf{u}^*, \qquad \left.\frac{\partial \phi}{\partial n}\right|_{wall} = 0, \tag{5}$$

discretised with the compact five-point operator (wall links removed for the Neumann condition) and solved by SOR ($\omega = 1.85$) to $\max|\delta\phi| < 10^{-8}$. The Neumann problem is solvable because $\oint \mathbf{u}^* \cdot \mathbf{n} = 0$ (discrete compatibility); ϕ is fixed up to a constant, which SOR selects through the zero initial guess.

3. **Corrector**: $\mathbf{u}^{n+1} = \mathbf{u}^* - \Delta t \nabla\phi,\ p^{n+1} = p^n + \phi$.

SMAC differs from the original MAC method in solving for the pressure *increment* ϕ rather than for p itself; at steady state $\phi \to 0$ and $\mathbf{u}^* \to \mathbf{u}^{n+1}$, so the converged solution satisfies the discrete continuity and momentum equations exactly (to the Poisson tolerance).

2.3 Stability and accuracy

The explicit predictor is limited by the convective (CFL) and viscous (diffusion number) conditions and by the cell Reynolds number of central differencing:

$$\mathrm{CFL} = \frac{U\Delta t}{\Delta x} = 0.099 < 1, \qquad d = \frac{\Delta t}{\mathrm{Re}\,\Delta x^2} = 0.098 < \tfrac{1}{4}, \qquad \mathrm{Re}_\Delta = \mathrm{Re}\,U\Delta x = 1.01 \lesssim 2. \tag{6}$$

The spatial discretisation is second-order; the time discretisation is first-order but irrelevant for the steady solution.

2.4 Algorithm and flowchart

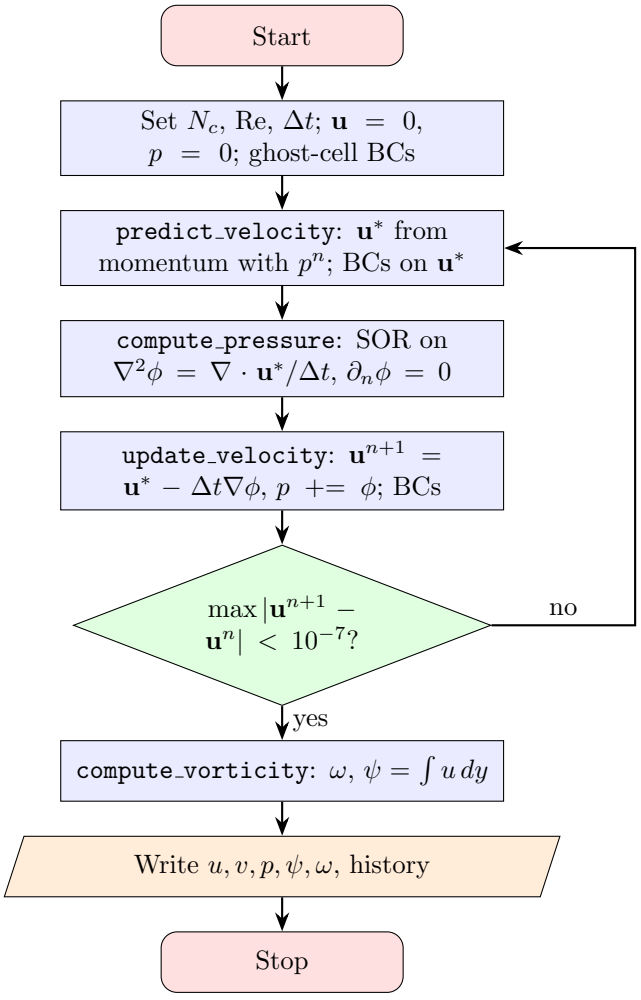


Figure 2: SMAC time-marching loop (`code/smac_cavity.c`).

3 Code review of the original submission

The original program (kept in `original/Assignment_3_original.c`) compiles with warnings and terminates with a **segmentation fault** (exit code 139) before writing any data; all five output files are empty. This is consistent with the original report, which states that the results could not be plotted. The defects were:

Location	Defect
`main: double S[i][j], Vor[i][j];`	Variable-length arrays sized by the *uninitialised* loop counters i, j (undefined behaviour): the cause of the crash.
`compute_pressure`	Not a pressure equation: p is updated with an ad-hoc combination of derivatives of u only ($h_x^2 + h_y^2 - h_{xx} - h_{yy} - h_{xy}$); v and the velocity divergence do not appear, so continuity is never enforced.
`update_velocity`	Only the pressure-gradient term is applied: no convection, no viscous term, i.e. no momentum equation. The results go to `unew`, `vnew`, which are never copied back, so u and v stay zero for all time.
boundary conditions	The wall/lid branches sit inside loops that run only over interior indices $(1 \dots N-2)$, so they never execute: the lid velocity is never imposed and nothing drives the flow.
`compute_vorticity`	Empty loop body.
#define `DX 1.0 / (NX - 1)`	Missing brackets: `a / DX` expands to `a / 1.0 / (NX-1)`.
`NT 1000`	$t_{end} = 1$: far from steady state (≈ 13.6 needed).
`float`	Single precision prevents converging the Poisson equation to small tolerances.

Table 1: Defects in the original code.

What was done. In this revision the program structure, constants (Re = 100, $\Delta t = 10^{-3}$, 100 grid lines per side), function names, time loop and output files were kept, and the function bodies were completed to implement the algorithm of Section 2. Additions are marked `[COMPLETED]`, `[FIX]` or `[ADDED]` in `code/smac_cavity.c`. Because the numerical core was written during this revision, the solver should be regarded as a completed version of the assignment rather than the original submission.

4 Results

4.1 Convergence to steady state

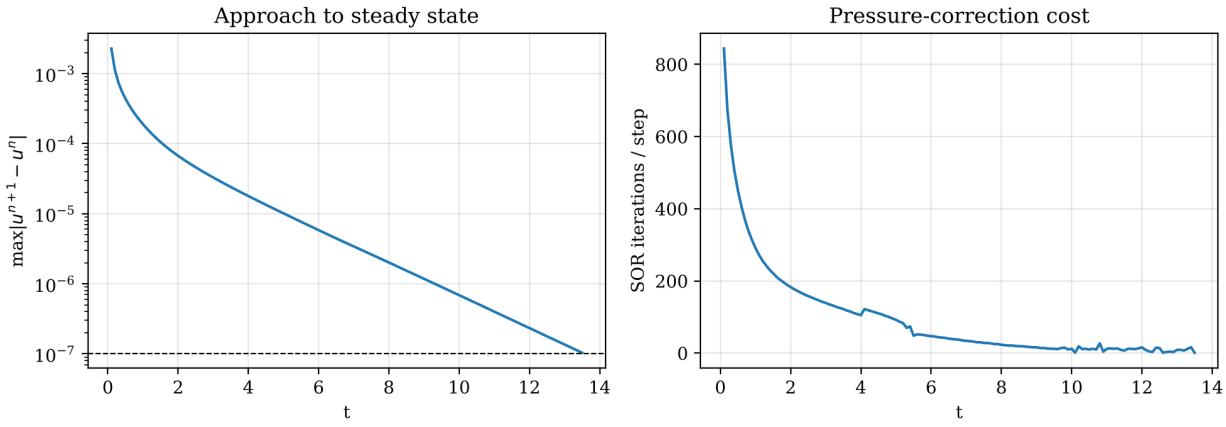


Figure 3: Maximum velocity change per step and SOR iterations per step for the 99×99 run.

The velocity change decays exponentially, at a rate set by the slowest viscous mode, and falls below 10^{-7} at step 13 563 ($t = 13.56$). The number of SOR sweeps per step drops from several hundred at start-up to about 10 as $\phi \to 0$, the expected behaviour of an incremental-pressure (SMAC) formulation. The run takes about 3.5 minutes on one CPU core.

4.2 Flow structure

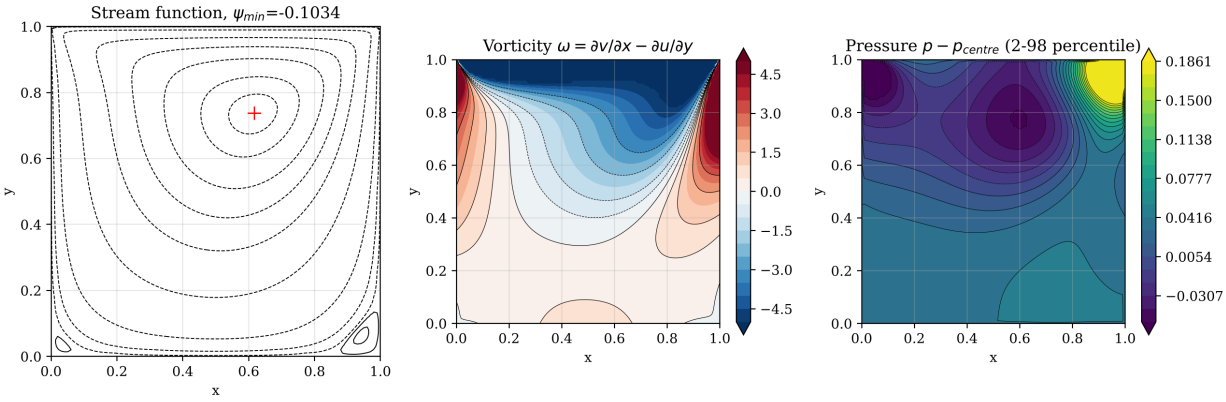


Figure 4: Stream function (primary vortex centre marked), vorticity and pressure for the 99×99 grid.

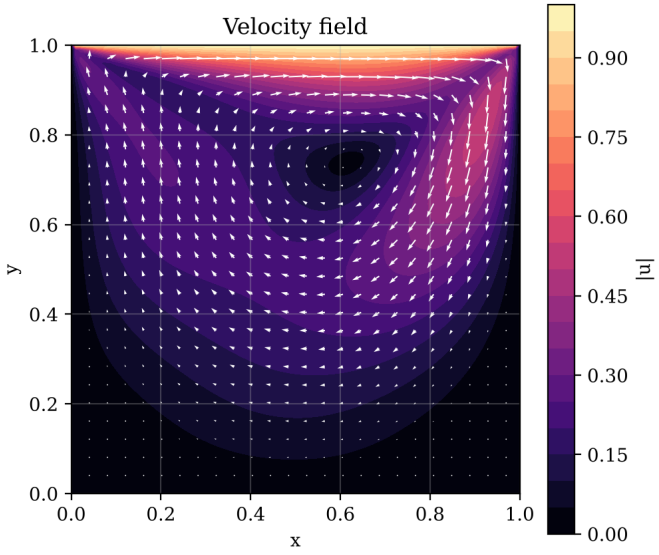


Figure 5: Velocity vectors over speed contours.

The flow consists of a primary clockwise vortex displaced towards the downstream (right) upper corner and two weak counter-rotating eddies in the bottom corners. The vorticity is concentrated in the lid boundary layer and is singular at the two lid corners, where the velocity jumps from 1 to 0; the downstream corner is a stagnation region of high pressure, and the pressure minimum coincides with the vortex core, where the radial pressure gradient balances the centripetal acceleration.

4.3 Validation against Ghia et al. (1982)

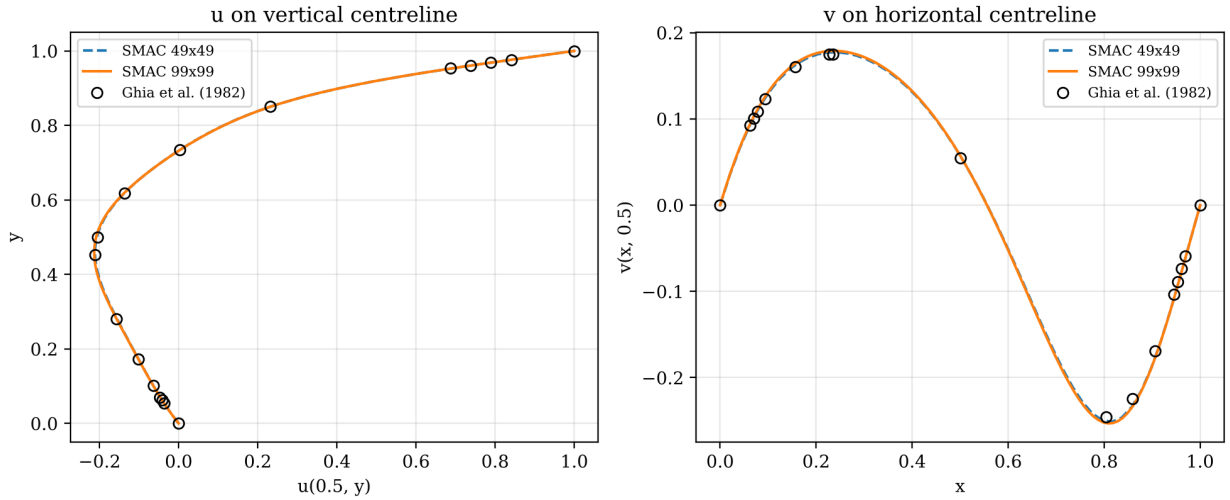


Figure 6: Centreline velocity profiles against the benchmark of Ghia, Ghia & Shin (1982), 129×129 multigrid stream function–vorticity solution.

Quantity	49×49	99×99	Ghia et al. (1982)
ψ_{min} (primary vortex)	-0.10294	-0.10337	-0.10342
Primary centre (x, y)	(0.612, 0.735)	(0.616, 0.737)	(0.617, 0.734)
ψ_{max} bottom-right eddy	1.62×10^{-5}	1.35×10^{-5}	1.25×10^{-5}
ψ_{max} bottom-left eddy	3.24×10^{-6}	2.13×10^{-6}	1.75×10^{-6}
u_{min} on $x = 0.5$	-0.2112	-0.2133	-0.2109
v_{max} / v_{min} on $y = 0.5$	0.1773 / -0.2513	0.1789 / -0.2531	0.1753 / -0.2453
$\max \lvert u - u_{Ghia} \rvert$ / rms	0.0048 / 0.0020	0.0049 / 0.0022	–
$\max \lvert v - v_{Ghia} \rvert$ / rms	0.0077 / 0.0037	0.0088 / 0.0044	–

Table 2: Comparison with the benchmark.

The primary vortex strength and position agree with the benchmark to 0.05% and within one cell. The secondary eddies, which are four to five orders of magnitude weaker, converge monotonically towards the benchmark values under refinement. The centreline discrepancies (≤ 0.009, i.e. $< 1\%$ of U) are of the same size as the uncertainty of the benchmark itself, which was computed on a 129×129 grid with a different formulation; later grid-converged studies report $u_{min} \approx -0.214$ at $\mathrm{Re} = 100$ (e.g. Marchi et al., 2009), which the 99×99 value (-0.2133) approaches from the coarse-grid side.

5 Conclusions

- The completed SMAC solver on a staggered grid reproduces the classical $\mathrm{Re} = 100$ cavity solution: $\psi_{min} = -0.1034$ at $(0.616, 0.737)$, both corner eddies, and centreline velocities within $< 1\%$ of the lid speed of Ghia et al.
- The staggered arrangement and the incremental pressure correction give a checkerboard-free pressure field and a Poisson cost that falls to ~ 10 SOR sweeps per step near steady state.
- The original code crashed (uninitialised variable-length arrays) and, even without the crash, contained no momentum equation, no continuity constraint and no lid boundary condition. Its structure was preserved and the numerical core completed; this is documented so the revised work can be presented accurately.

References

U. Ghia, K.N. Ghia, C.T. Shin, High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method, *J. Comput. Phys.* 48 (1982) 387–411.

A.A. Amsden, F.H. Harlow, A simplified MAC technique for incompressible fluid flow calculations, *J. Comput. Phys.* 6 (1970) 322–325.

F.H. Harlow, J.E. Welch, Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface, *Phys. Fluids* 8 (1965) 2182–2189.

C.H. Marchi, R. Suero, L.K. Araki, The lid-driven square cavity flow: numerical solution with a 1024×1024 grid, *J. Braz. Soc. Mech. Sci. Eng.* 31 (2009) 186–198.

Reproducing the results

`bash run.sh` builds and runs the 99×99 and 49×49 cases and calls `code/postprocess.py`.

A Source code

```c
// 2-D lid-driven cavity, incompressible Navier-Stokes, SMAC (Simplified Marker-And-Cell) method
// Original author: Karthick J (OE23S018), AM5630 Computer Assignment 3.
// The original program kept this structure (constants, compute_pressure, update_velocity,
// compute_vorticity, time loop, output files) but the function bodies did not solve the
// Navier-Stokes equations (see report, Section 3). The bodies were COMPLETED here following the
// SMAC algorithm stated in the original report; all completed parts are marked [COMPLETED]/[FIX].
//
// Grid: staggered MAC grid with NC x NC cells (NC = NX-1): p at cell centres, u on vertical
// faces, v on horizontal faces, one ghost layer for the wall conditions.
// SMAC step: (1) u* = u^n + dt[-(u.grad)u + (1/Re) lap(u) - grad p^n]
// (2) lap(phi) = div(u*)/dt (Neumann BC, SOR)
// (3) u^{n+1} = u* - dt grad(phi), p^{n+1} = p^n + phi

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Define constants
#define NX 100 // grid points per side (as original); NC = NX-1 = 99 cells
#define NY 100
#define NC (NX - 1)
#define NT 200000 // [FIX: max time steps; was 1000 (t = 1), far from steady state]
#define DX (1.0 / (NX - 1)) // [FIX: brackets added; '1.0 / (NX - 1)' without them breaks in expressions]
#define DY (1.0 / (NY - 1))
#define DT 0.001
#define RE 100.0
#define U_INFTY 1.0
#define STEADY_TOL 1.0e-7 // [ADDED] steady state: max|u^{n+1}-u^n|/U < tol
#define SOR_OMEGA 1.85 // [ADDED] SOR factor for the pressure-correction Poisson equation
#define SOR_TOL 1.0e-8 // [ADDED]

// Define variables [FIX: double precision; float cannot converge the Poisson solve to 1e-8]
double u[NC + 1][NC + 2], v[NC + 2][NC + 1], p[NC + 2][NC + 2];
double unew[NC + 1][NC + 2], vnew[NC + 2][NC + 1], pnew[NC + 2][NC + 2]; // u*, v*, phi
double S[NC + 1][NC + 1], Vor[NC + 1][NC + 1]; // [FIX: were VLAs sized by uninitialised i, j]

// [COMPLETED] wall conditions through ghost cells: no slip on all walls, u = U_INFTY on the lid
void apply_velocity_bc(double uu[NC + 1][NC + 2], double vv[NC + 2][NC + 1]) {
    for (int i = 0; i <= NC; i++) {
        uu[i][0] = -uu[i][1]; // bottom wall: u = 0 at y = 0
        uu[i][NC + 1] = 2.0 * U_INFTY - uu[i][NC]; // top lid: u = U at y = 1
    }
    for (int j = 0; j <= NC + 1; j++) { uu[0][j] = 0.0; uu[NC][j] = 0.0; } // left/right walls
    for (int j = 0; j <= NC; j++) {
        vv[0][j] = -vv[1][j]; // left wall: v = 0 at x = 0
        vv[NC + 1][j] = -vv[NC][j]; // right wall: v = 0 at x = 1
    }
    for (int i = 0; i <= NC + 1; i++) { vv[i][0] = 0.0; vv[i][NC] = 0.0; } // bottom/top walls
}

// Function to compute pressure with boundary conditions
// [COMPLETED] SMAC pressure correction: lap(phi) = div(u*)/DT, dphi/dn = 0 on walls (SOR)
int compute_pressure() {
    int i, j, it;
    for (i = 0; i <= NC + 1; i++) for (j = 0; j <= NC + 1; j++) pnew[i][j] = 0.0;
    for (it = 1; it <= 20000; it++) {
        double res = 0.0;
        for (i = 1; i <= NC; i++) {
            for (j = 1; j <= NC; j++) {
                double div = (unew[i][j] - unew[i - 1][j]) / DX + (vnew[i][j] - vnew[i][j - 1]) / DY;
                double aw = (i > 1) ? 1.0 : 0.0, ae = (i < NC) ? 1.0 : 0.0; // Neumann walls:
                double as = (j > 1) ? 1.0 : 0.0, an = (j < NC) ? 1.0 : 0.0; // drop wall links
                double ap = (aw + ae) / (DX * DX) + (as + an) / (DY * DY);
                double rhs = (aw * pnew[i - 1][j] + ae * pnew[i + 1][j]) / (DX * DX)
                           + (as * pnew[i][j - 1] + an * pnew[i][j + 1]) / (DY * DY) - div / DT;
                double r = rhs / ap - pnew[i][j];
                pnew[i][j] += SOR_OMEGA * r;
                if (fabs(r) > res) res = fabs(r);
            }
        }
        if (res < SOR_TOL) break;
    }
    return it;
}
```

```
75
76  // Function to update velocity with boundary conditions
77  // [COMPLETED] (a) predictor u*, v* from the momentum equations (explicit Euler, central differences)
78  void predict_velocity() {
79      int i, j;
80      for (i = 1; i < NC; i++) {
81          for (j = 1; j <= NC; j++) {
82              double ue = 0.5 * (u[i][j] + u[i + 1][j]), uw = 0.5 * (u[i - 1][j] + u[i][j]);
83              double un = 0.5 * (u[i][j] + u[i][j + 1]), us = 0.5 * (u[i][j - 1] + u[i][j]);
84              double vn = 0.5 * (v[i][j] + v[i + 1][j]), vs = 0.5 * (v[i][j - 1] + v[i + 1][j - 1]);
85              double conv = (ue * ue - uw * uw) / DX + (un * vn - us * vs) / DY;
86              double diff = (u[i + 1][j] - 2 * u[i][j] + u[i - 1][j]) / (DX * DX)
87                          + (u[i][j + 1] - 2 * u[i][j] + u[i][j - 1]) / (DY * DY);
88              unew[i][j] = u[i][j] + DT * (-conv + diff / RE - (p[i + 1][j] - p[i][j]) / DX);
89          }
90      }
91      for (i = 1; i <= NC; i++) {
92          for (j = 1; j < NC; j++) {
93              double vn = 0.5 * (v[i][j] + v[i][j + 1]), vs = 0.5 * (v[i][j - 1] + v[i][j]);
94              double ve = 0.5 * (v[i][j] + v[i + 1][j]), vw = 0.5 * (v[i - 1][j] + v[i][j]);
95              double ue = 0.5 * (u[i][j] + u[i][j + 1]), uw = 0.5 * (u[i - 1][j] + u[i - 1][j + 1]);
96              double conv = (ue * ve - uw * vw) / DX + (vn * vn - vs * vs) / DY;
97              double diff = (v[i + 1][j] - 2 * v[i][j] + v[i - 1][j]) / (DX * DX)
98                          + (v[i][j + 1] - 2 * v[i][j] + v[i][j - 1]) / (DY * DY);
99              vnew[i][j] = v[i][j] + DT * (-conv + diff / RE - (p[i][j + 1] - p[i][j]) / DY);
100         }
101     }
102     apply_velocity_bc(unew, vnew);
103 }
104
105 // [COMPLETED] (b) corrector: project u* onto a divergence-free field, update pressure.
106 // Returns max |u^{n+1} - u^n| for the steady-state test. [FIX: original never copied unew -> u]
107 double update_velocity() {
108     int i, j; double change = 0.0;
109     for (i = 1; i < NC; i++) for (j = 1; j <= NC; j++) {
110         double un1 = unew[i][j] - DT * (pnew[i + 1][j] - pnew[i][j]) / DX;
111         if (fabs(un1 - u[i][j]) > change) change = fabs(un1 - u[i][j]);
112         u[i][j] = un1;
113     }
114     for (i = 1; i <= NC; i++) for (j = 1; j < NC; j++) {
115         double vn1 = vnew[i][j] - DT * (pnew[i][j + 1] - pnew[i][j]) / DY;
116         if (fabs(vn1 - v[i][j]) > change) change = fabs(vn1 - v[i][j]);
117         v[i][j] = vn1;
118     }
119     for (i = 1; i <= NC; i++) for (j = 1; j <= NC; j++) p[i][j] += pnew[i][j];
120     apply_velocity_bc(u, v);
121     return change;
122 }
123
124 // Function to compute vorticity
125 // [COMPLETED] vorticity at cell corners: w = dv/dx - du/dy; stream function from integrating
126 // u = dpsi/dy along y (psi = 0 on all walls)
127 void compute_vorticity() {
128     int i, j;
129     for (i = 0; i <= NC; i++) {
130         for (j = 0; j <= NC; j++) {
131             Vor[i][j] = (v[i + 1][j] - v[i][j]) / DX - (u[i][j + 1] - u[i][j]) / DY;
132         }
133         S[i][0] = 0.0;
134         for (j = 1; j <= NC; j++) S[i][j] = S[i][j - 1] + u[i][j] * DY;
135     }
136 }
137
138 int main() {
139     int i, j, t;
140     FILE *streamlineFile, *vorticityFile, *pressureFile, *uFile, *vFile, *histFile;
141
142     // Open files for writing
143     streamlineFile = fopen("streamlines.txt", "w");
144     vorticityFile = fopen("vorticity.txt", "w");
145     pressureFile = fopen("pressure.txt", "w");
146     uFile = fopen("u_values.txt", "w");
147     vFile = fopen("v_values.txt", "w");
148     histFile = fopen("convergence_history.txt", "w"); // [ADDED]
149
```

```
    if (streamlineFile == NULL || vorticityFile == NULL || pressureFile == NULL || uFile == NULL || vFile ==
         NULL) {
        printf("Error opening files.\n");
        return 1; // Exit the program if the file opening fails
    }

    // Initialize the velocity and pressure fields
    for (i = 0; i <= NC; i++) for (j = 0; j <= NC + 1; j++) u[i][j] = 0.0;
    for (i = 0; i <= NC + 1; i++) for (j = 0; j <= NC; j++) v[i][j] = 0.0;
    for (i = 0; i <= NC + 1; i++) for (j = 0; j <= NC + 1; j++) p[i][j] = 0.0;
    apply_velocity_bc(u, v);
    printf("SMAC lid-driven cavity: Re=%.0f, %dx%d cells, dt=%g, CFL=%.3f, diffusion number=%.3f\n",
           RE, NC, NC, DT, U_INFTY * DT / DX, DT / (RE * DX * DX));

    // Time-stepping loop
    double change = 1.0;
    for (t = 1; t <= NT; t++) {
        predict_velocity();
        int sor_it = compute_pressure();
        change = update_velocity();
        if (t % 100 == 0) {
            fprintf(histFile, "%d %.6f %.6e %d\n", t, t * DT, change, sor_it);
            if (t % 2000 == 0) printf("step %6d t=%7.3f max|du|=%.3e SOR its=%d\n", t, t * DT, change, sor_it);
        }
        if (change < STEADY_TOL) { printf("Steady state reached at step %d (t = %.3f)\n", t, t * DT); break; }
    }
    compute_vorticity();

    /// Write values to files [FIX: values interpolated to the (NC+1)x(NC+1) cell-corner nodes;
    /// columns: x y value, one block per y-row]
    for (j = 0; j <= NC; j++) {
        for (i = 0; i <= NC; i++) {
            double x = i * DX, y = j * DY;
            double uc = 0.5 * (u[i][j] + u[i][j + 1]);
            double vc = 0.5 * (v[i][j] + v[i + 1][j]);
            double pc = 0.25 * (p[i][j] + p[i + 1][j] + p[i][j + 1] + p[i + 1][j + 1]);
            fprintf(streamlineFile, "%f %f %e\n", x, y, S[i][j]);
            fprintf(vorticityFile, "%f %f %e\n", x, y, Vor[i][j]);
            fprintf(pressureFile, "%f %f %e\n", x, y, pc);
            fprintf(uFile, "%f %f %e\n", x, y, uc);
            fprintf(vFile, "%f %f %e\n", x, y, vc);
        }
    }

    // Close files
    fclose(streamlineFile);
    fclose(vorticityFile);
    fclose(pressureFile);
    fclose(uFile);
    fclose(vFile);
    fclose(histFile);

    return 0;
}
```